Operating Systems

[1500WETOPS]

José Oramas

Concurrency

[Ensuring Coordinated Functioning of Multiple Processes]

José Oramas

University of Antwerp

PSA

Today's Session:

- Around one hour (up to 15h00)
- What to do with part-2 of the session?
  - ***Option-A:*** Self-study at home & conclude the theory on 08/12
  - ***Option-B:*** Continue next week & conclude the theory on 15/12

PSA

Today's Session:

- Around one hour (up to 15h00)
- What to do with part-2 of the session?
  - ***Option-A:*** Self-study at home & conclude the theory on 08/12
  - ***Option-B:*** Continue next week & conclude the theory on 15/12

Next Exercise Session (Concurrency): 16/12

- All related content will be released today

Concurrency

Let’s consider the following script

```
procedure echo;
var outp, inp: character;
begin
    input (inp, keyboard);
    outp := inp;
    output (outp,display);
end
```

Let’s assume

- Several processes will use it

  → We put it in memory so that it can be shared

- Processes P0 and P1 access the script.

Concurrency

Introduction

- Processes that share resources can result in incorrect program flow
- Prevention via **Mutual Exclusion** within critical sections
- Possible problems:
  - starvation (livelock)
  - deadlocks

Concurrency

Existing Approaches

- **Software**

  Processes must take care of mutual exclusion themselves

- **Hardware**

  Special hardware instructions to support mutual exclusion

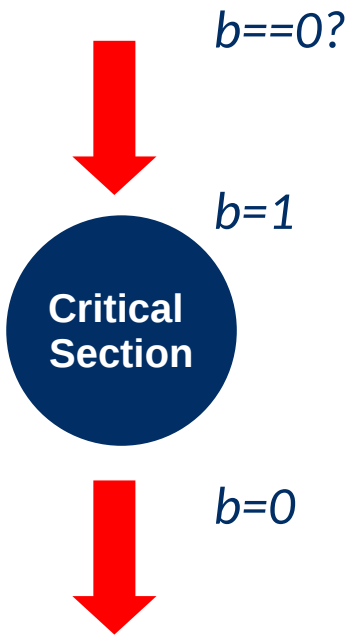
- OS and programming language support

Software-Based Solutions

Concurrency

Possible solution:

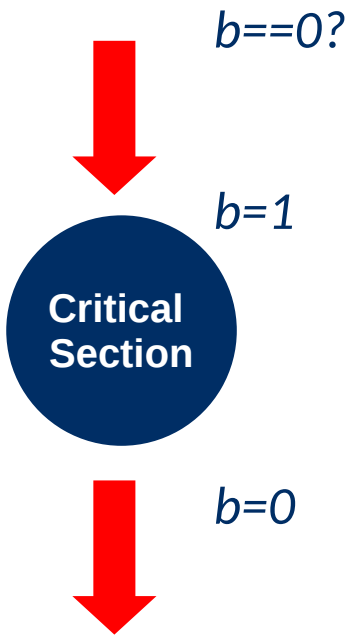
- **Bit per critical section**
  - For each critical section keep a variable to indicated whether a process is present
  - Set it to 1 on entry, to 0 on exit
  - test if(bit=0) before we enter.



Concurrency

Possible solution:

- **Bit per critical section**
  - For each critical section keep a variable to indicated whether a process is present
  - Set it to 1 on entry, to 0 on exit
  - test if(bit=0) before we enter.



Issues

- Testing at 0 and setting equal to 1 is not an **atomic operation**,
  → process can be interrupted between testing and setting the bit
- If CS is occupied, then constant checking is required → ***busy waiting***

Software-Based Solutions

Dekker's Algorithm – 1st Attempt

- Use a global variable ***turn*** to indicate which of the two processes may use the CS next.

PROCESS 0

```
..
while turn ≠ 0 do {nothing};
<critical section>
turn := 1;
..
```

PROCESS 1

```
..
while turn ≠ 1 do {nothing};
<critical section>
turn := 0;
..
```

Software-Based Solutions

Dekker's Algorithm – 1st Attempt

- Use a global variable ***turn*** to indicate which of the two processes may use the CS next.

PROCESS 0

```
..
while turn ≠ 0 do {nothing};
<critical section>
turn := 1;
..
```

PROCESS 1

```
..
while turn ≠ 1 do {nothing};
<critical section>
turn := 0;
..
```

Issues

- Iterative change of *turn* → faster process gets delayed
- Initialization of the *turn* variable will favor one of the processes.

Software-Based Solutions

Dekker's Algorithm – 2nd Attempt

- Keep a variable for each process → **flag[0]** & **flag[1]**
- To enter the CS first check whether the other process has the intention to enter

PROCESS 0

```
..
while flag[1] do {nothing};
flag[0]:=true;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
while flag[0] do {nothing};
flag[1]:=true;
<critical section>
flag[1]:=false;
..
```

Software-Based Solutions

Dekker's Algorithm – 2nd Attempt

- Keep a variable for each process → **flag[0]** & **flag[1]**
- To enter the CS first check whether the other process has the intention to enter

PROCESS 0

```
..
while flag[1] do {nothing};
flag[0]:=true;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
while flag[0] do {nothing};
flag[1]:=true;
<critical section>
flag[1]:=false;
..
```

Issues

- What if a process gets interrupted after the while loop, the other process will enter as well

  → no mutual exclusion

Software-Based Solutions

Dekker's Algorithm – 3rd Attempt

- 2nd Attempt + switching the assignment and while lines

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do {nothing};
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do {nothing};
<critical section>
flag[1]:=false;
..
```

Software-Based Solutions

Dekker's Algorithm – 3rd Attempt

- 2nd Attempt + switching the assignment and while lines

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do {nothing};
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do {nothing};
<critical section>
flag[1]:=false;
..
```

Issues

- If both process manage to flag[i] = true, then both will be stuck in the while loop.

  → deadlock

Software-Based Solutions

Dekker's Algorithm – 4th Attempt

- 3rd Attempt + Pause[if another process has intention, our flag is set to 0 and we wait]

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do
   flag[0]:=false;
   <delay a short time>
   flag[0]:=true;
end;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do
   flag[1]:=false;
   <delay a short time>
   flag[1]:=true;
end;
<critical section>
flag[1]:=false;
..
```

Software-Based Solutions

Dekker's Algorithm – 4th Attempt

- 3rd Attempt + Pause[if another process has intention, our flag is set to 0 and we wait]

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do
   flag[0]:=false;
   <delay a short time>
   flag[0]:=true;
end;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do
   flag[1]:=false;
   <delay a short time>
   flag[1]:=true;
end;
<critical section>
flag[1]:=false;
..
```

Issues

- The waiting time is not bounded

Software-Based Solutions

Dekker's Algorithm – Final Solution

- Use a single variable ***turn*** to indicate which process will be courteous when in conflict. E.g., if (*turn* == 1) → P0 is courteous.

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do
 if turn = 1 then
    flag[0]:=false;
    while turn = 1 do {nothing};
    flag[0]:=true;
 end;
end;
<critical section>
turn:=1;
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do
 if turn = 0 then
    flag[1]:=false;
    while turn = 0 do {nothing};
    flag[1]:=true;
 end;
end;
<critical section>
turn:=0;
flag[1]:=false;
..
```

Software-Based Solutions

Dekker's Algorithm – Final Solution

- Use a single variable ***turn*** to indicate which process will be courteous when in conflict. E.g., if (*turn* == 1) → P0 is courteous.

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do
 if turn = 1 then
    flag[0]:=false;
    while turn = 1 do {nothing};
    flag[0]:=true;
 end;
end;
<critical section>
turn:=1;
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do
 if turn = 0 then
    flag[1]:=false;
    while turn = 0 do {nothing};
    flag[1]:=true;
 end;
end;
<critical section>
turn:=0;
flag[1]:=false;
..
```

Note

- Deadlocks are prevented due to the conditioning from the *flag* and *turn* variables on the while loop

Software-Based Solutions

Dekker's Algorithm

- **Overall Properties**
  - *Busy waiting* occurs
  - Priority for process that entered the critical section least recently
  - Not trivial to generalize to *n* processes.

Software-Based Solutions

Peterson's Algorithm

- Announce desire to enter the CS
- Set the turn to the other process
- Verify whether P1 does not have intention or is in the CS.

PROCESS 0

```
..
flag[0]:=true;
turn = 1;
while flag[1] and turn = 1 do {nothing };
<critical section>
flag[0]:=false;
..
```

Note

- Can be generalized to n processes

Software-Based Solutions

Peterson's Algorithm – *n* Processes

- Assume each process crosses *n-1* phases before reaching the CS.

```
PROCESS i
..
for j = 1 to n-1 do
    flag[i]:= j;
    turn[j] := i;
    wait until (∀ k ≠ i, flag[k] < j) or (turn[j] ≠ i);
end;
<critical section>
flag[i]:=0;
..
```

Variables:

- *Flag[i]:* phase of P_i
- *Turn[j]:* which was the last process in phase j.

Software-Based Solutions

Peterson's Algorithm – *n* Processes

- Assume each process crosses *n-1* phases before reaching the CS.

```
PROCESS i
..
for j = 1 to n-1 do
    flag[i]:= j;
    turn[j] := i;
    wait until (∀ k ≠ i, flag[k] < j) or (turn[j] ≠ i);
end;
<critical section>
flag[i]:=0;
..
```

Variables:

- *Flag[i]:* phase of P_i
- *Turn[j]:* which was the last process in phase j.

Software-Based Solutions

Peterson's Algorithm – *n* Processes

- Assume each process crosses *n-1* phases before reaching the CS.

```
PROCESS i
..
for j = 1 to n-1 do
    flag[i]:= j;
    turn[j] := i;
    wait until (∀ k ≠ i, flag[k] < j) or (turn[j] ≠ i);
end;
<critical section>
flag[i]:=0;
..
```

Variables:

- *Flag[i]:* phase of P_i
- *Turn[j]:* which was the last process in phase j.

Q: Does *busy waiting* still occurs?

Hardware-Based Solutions

Hardware-Base Solutions

Characteristics

- Prohibit interrupts during critical section
  - Disadvantageous for large critical sections

Hardware-Base Solutions

Characteristics

- Prohibit interrupts during critical section
  - Disadvantageous for large critical sections
- Machine instruction support (spinlock)
  - *Test-and-set*
    - bit per critical section
  - *Exchange*
    - *bolt=1* and for each process *key=0*, exchange from *(key,bolt)* to *key=1*
  - Cons: busy waiting, no guarantees against starvation

PROCESS i

..
while (testset(bolt)) **do** {nothing};
<critical section>
bolt:=false;
..

Hardware-Base Solutions

Semaphore

- Introduced by Dijkstra
- (Counting) Semaphore ***s*** a is record:
  - ***count***: integer (initially not negative)
  - **wait(s)**: decrease counter with one, if the counter becomes negative then we block the process that performed the *wait(s)* operation (in FIFO list)
  - **signal(s)**: increase counter by one, if obtained value is not positive, oldest already blocked process is unblocked

Hardware-Base Solutions

Semaphore

wait(s);

```
s.count := s.count - 1;
if s.count < 0
  then begin
    place this process in s.queue;
    block this process
  end;
```

signal(s);

```
s.count := s.count + 1;
if s.count ≤ 0
  then begin
    remove a process P from s.queue;
    place process P on the ready list
  end;
```

Hardware-Base Solutions

Binary Semaphore

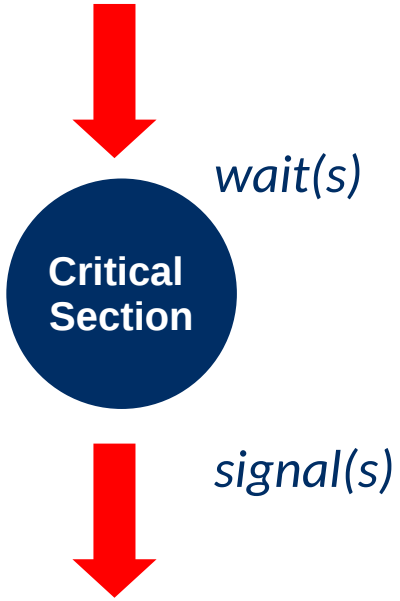
- Counter (*count*) only accepts 0 or 1 as a value
  - *waitB(s)*: check if counter is 1, if so we lower it, otherwise we block it
  - *signalB(s)*: if no processes are blocked, then set the counter to one, otherwise unblock one process (oldest)

- Binary semaphores are as powerful as counting semaphores

Hardware-Base Solutions

Mutual Exclusion with Semaphore

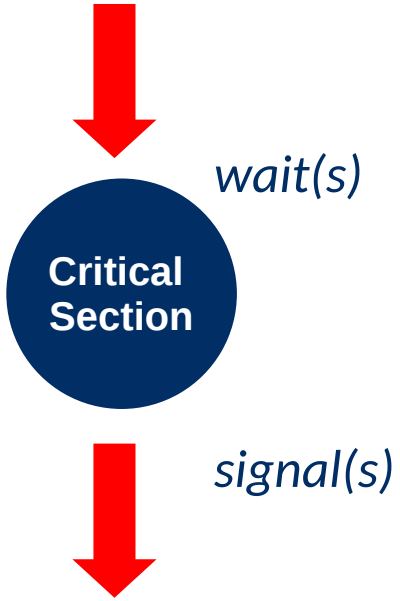
- Semaphore per CS
  - Initializing semaphore with one
  - Entering CS → *wait(s)*
  - Exit CS → *signal(s)*



Hardware-Base Solutions

Mutual Exclusion with Semaphore

- Semaphore per CS
  - Initializing semaphore with one
  - Entering CS → *wait(s)*
  - Exit CS → *signal(s)*



[wait(s) and *signal(s)* operations cannot be interrupted or occur simultaneously by different processes]

Hardware-Based Solutions

Implementing Semaphores

- Wait(s) and *signal(s)* must be atomic operations or cannot be executed simultaneously by different processes

Hardware-Based Solutions

Implementing Semaphores

- Wait(s) and *signal(s)* must be atomic operations or cannot be executed simultaneously by different processes
  - Via harware solution
    - *test-and-set* or *exchange*
    - *wait(s)* and *signal(s)* must be concise → ~ dozen instructions
      - List defined by pointers in the process control block.

Hardware-Based Solutions

Implementing Semaphores

- Wait(s) and *signal(s)* must be atomic operations or cannot be executed simultaneously by different processes
  - Via harware solution
    - *test-and-set* or *exchange*
    - *wait(s)* and *signal(s)* must be concise → ~ dozen instructions
      - List defined by pointers in the process control block.

    Disadvantages:

    - busy-waiting, however for short ops this is sometimes faster than real process swap
    - Interruptions also interrupt (disabling interruptions is sufficient with single processor)

Hardware-Based Solutions

Producers / Consumers Problem

- One or more processes “produce” data and write it to a buffer.
- One process access this data by reading it from the buffer.

Hardware-Based Solutions

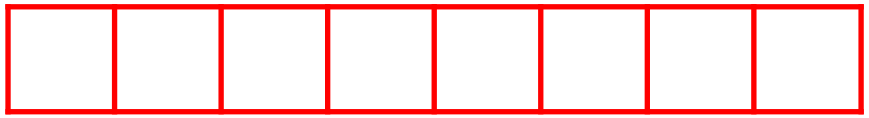
Producers / Consumers Problem

- One or more processes “produce” data and write it to a buffer.
- One process access this data by reading it from the buffer.

Challenges:

- Can only take something if it's there
- May take item only once
- Each produced item must have a new place

Hardware-Based Solutions



Producers / Consumers Problem

- A possible implementation

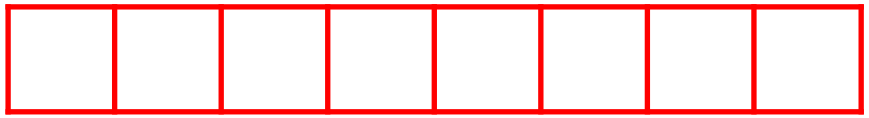
Producer:

```
repeat
    produce item v;
    b[in] := v;
    in := in +1;
forever;
```

Consumer:

```
repeat
    while in ≤ out do {nothing};
    w := b[out];
    out := out +1;
    consume item w;
forever;
```

Hardware-Based Solutions



Producers / Consumers Problem

- A possible implementation

Producer:

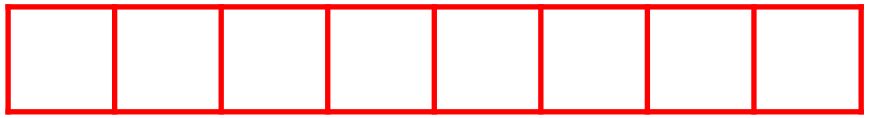
```
repeat
    produce item v;
    b[in] := v;
    in := in +1;
forever;
```

Consumer:

```
repeat
    while in ≤ out do {nothing};
    w := b[out];
    out := out +1;
    consume item w;
forever;
```

Q: What would be needed to ensure proper concurrency?

Hardware-Based Solutions



Producers / Consumers Problem

- A possible implementation

Producer:

```
repeat
    produce item v;
    b[in] := v;
    in := in +1;
forever;
```

Consumer:

```
repeat
    while in ≤ out do {nothing};
    w := b[out];
    out := out +1;
    consume item w;
forever;
```

Note:

- We need to protect the variables *in* and *out*
- Prevent the buffer from being called by several processes at the same time.
- The buffer may not be read when empty.

Hardware-Based Solutions

Producers / Consumers Problem – Binary Semaphores

- A possible implementation (assuming *n = in – out*)

Producer:

```
repeat
    produce;
    waitB(s);
    append;
    n := n +1;
    if n = 1 then signalB(ne);
    signalB(s);
forever;
```

Consumer:

```
waitB(ne);
repeat
    waitB(s);
    take;
    n := n - 1;
    signalB(s);
    if n = 0 then waitB(ne);
    consume;
forever;
```

Hardware-Based Solutions

Producers / Consumers Problem – Binary Semaphores

- A possible implementation (assuming *n = in – out*)

Producer:

```
repeat
    produce;
    waitB(s);
    append;
    n := n +1;
    if n = 1 then signalB(ne);
    signalB(s);
forever;
```

Consumer:

```
waitB(ne);
repeat
    waitB(s);
    take;
    n := n - 1;
    signalB(s);
    if n = 0 then waitB(ne);
    consume;
forever;
```

Q: Is this implementation applicable when multiple consumer are active?

Hardware-Based Solutions

Producers / Consumers Problem – General/Counting Semaphores

- A possible implementation

Producer:

```
repeat
    produce;
    wait(s);
    append;
    signal(s);
    signal(ne);
forever;
```

Consumer:

```
repeat
    wait(ne);
    wait(s);
    take;
    signal(s);
    consume;
forever;
```

Hardware-Based Solutions

Producers / Consumers Problem – General/Counting Semaphores

- A possible implementation

Producer:

```
repeat
    produce;
    wait(s);
    append;
    signal(s);
    signal(ne);
forever;
```

Consumer:

```
repeat
    wait(ne);
    wait(s);
    take;
    signal(s);
    consume;
forever;
```

Q: What would happen if we switch the two *wait* operations of the Consumer?

Hardware-Based Solutions

Readers / Writers Problem

- A number of *readers* want to read data (e.g. catalogue), but the data can be modified by *writers*.
- Readers
  - Simultaneous access is allowed
  - No writers allowed
- Writers
  - A maximum of one writer is always active
  - No readers may be active

Hardware-Based Solutions

Readers / Writers through Semaphores

- How many readers are active?

- Stop writers when readers are active

- Stop readers when writer is active

Hardware-Based Solutions

Readers / Writers through Semaphores

- How many readers are active?
  - *r.count*: variable the number of active readers
  - semaphore *x* enforcing mutual exclusion for *r.count*
- Stop writers when readers are active

- Stop readers when writer is active

Hardware-Based Solutions

Readers / Writers through Semaphores

- How many readers are active?
  - *r.count*: variable the number of active readers
  - semaphore *x* enforcing mutual exclusion for *r.count*
- Stop writers when readers are active
  - *wsem:* prevents writers as long as there are readers, if *r.count* decreases to zero then *signal(wsem)*
- Stop readers when writer is active

Hardware-Based Solutions

Readers / Writers through Semaphores

- How many readers are active?
  - *r.count*: variable the number of active readers
  - semaphore *x* enforcing mutual exclusion for *r.count*
- Stop writers when readers are active
  - *wsem:* prevents writers as long as there are readers, if *r.count* decreases to zero then *signal(wsem)*
- Stop readers when writer is active
  - wsem: if r.count to one then *wait(wsem)*

Hardware-Based Solutions

Readers / Writers through Semaphores

- Implementation

Procedure: reader

```
wait(x);
   rcount := rcount+1;
   if rcount = 1 then wait(wsem);
signal(x);
READ
wait(x);
   rcount := rcount-1;
   if rcount = 0 then signal(wsem);
signal(x);
```

Procedure: writer

```
wait(wsem);
   WRITE;
signal(wsem);
```

Hardware-Based Solutions

Readers / Writers through Semaphores with Priority Writers

- Implementation

```
procedure reader
 wait(rsem);
  wait(x);
    rcount := rcount+1;
    if rcount = 1 then wait(wsem);
  signal(x);
 signal(rsem);
READ
wait(x);
 rcount := rcount-1;
 if rcount = 0 then signal(wsem);
signal(x);
```

```
procedure writer:
 wait(y);
    wcount := wcount+1;
    if wcount = 1 then wait(rsem);
 signal(y);
 wait(wsem);
    WRITE;
 signal(wsem);
 wait(y);
    wcount := wcount-1;
    if wcount = 0 then signal(rsem);
 signal(y);
```

Key points

Concurrence

Pay attention to…

- Stepwise solution to Dekker's algorithm, & Peterson’s Algorithm
- Hardware support
- Semaphores(types, use for mutual exclusion, implementation,
- Producer/Consumer & Readers/Writers Problems, and related solutions.

Further Reading

Essential: Concurrency

- OS Course Notes - chapter 7.
- Blackboard – Recorded lectures from Prof. Van Houdt.
  - Concurrency, deel I
  - Concurrency, Producers/Consumers Problem
  - Concurrency, Readers/Writers Problem

Optional

- Deadlocks: in lectures slide from previous academic years.

Questions?

Operating Systems

[1500WETOPS]

José Oramas